

Module-4 Part-1

Position and Orientation - Representing robot position. Basics of reactive navigation; Robot Localization, Challenges in localization - An error model for odometric position estimation

Navigation in Mobile Robots

- Navigation means **A robot moving from one place to another safely and intelligently to achieve a goal.**

For example:

- A delivery robot moving inside a hospital
- A warehouse robot going to pick up a box
- A self-driving car reaching a destination
- Navigation is difficult because the robot must understand the world and control itself correctly.
- Where am I? → Localization
- Where should I go? → Path Planning
- How do I move safely? → Obstacle Avoidance / Control

The Four Building Blocks of Navigation

- Navigation is one of the most challenging competences required of a mobile robot.
- Success in navigation requires success at the four building blocks of navigation:
 1. ***Perception*** (the robot must interpret its sensors to extract meaningful data).
 2. ***Localization*** (the robot must determine its position in the environment).
 3. ***Cognition*** (the robot must decide how to act to achieve its goals).
 4. ***Motion control*** (the robot must modulate its motor outputs to achieve the desired trajectory).

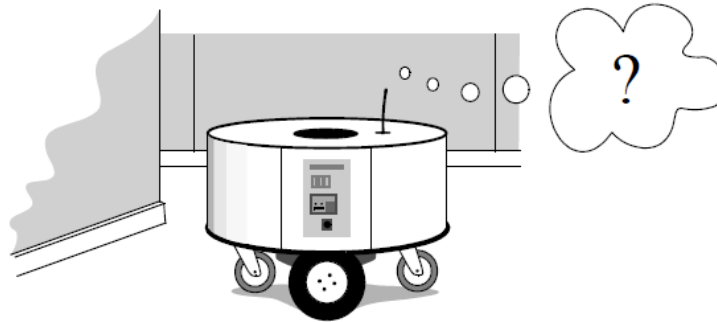


Figure 5.1
Where am I?

Perception: understanding what is happening

- The robot uses sensors like:
 - Camera
 - LiDAR
 - Ultrasonic sensors
 - GPS
- It must convert raw sensor data into meaningful information.
- Example:
 - Detect walls
 - Identify obstacles
 - Recognize landmarks

Without perception, the robot is blind.

Localization: Know where I am

- Localization means The robot must determine its position in the environment.
- The robot estimates:
 - Its x position
 - Its y position
 - Its orientation θ

Localization is difficult

- Because:
 - Sensors are noisy (The measurements change slightly every time.)
 - Wheels slip (instead of 1 meter, it moves 0.92m, due to slippery dusty floor)
 - Measurements are imperfect (small errors in Angle measurements ,Encoder readings ,GPS signals)
- That is why localization has received **the most research attention** in robotics.

- The most commonly used localization methods in modern robotics:
 - Extended Kalman Filter (EKF)
 - Particle Filter (Monte Carlo Localization)
 - SLAM algorithms
- If the robot does not know its position:
 - It cannot plan correctly
 - It cannot reach its goal
 - It cannot update its map
- So localization is the foundation of intelligent navigation.

Cognition: Decide where to go

- This is the decision-making part.
- The robot must:
 - Plan a path
 - Avoid obstacles
 - Choose actions to reach the goal
- Example:
 - Should I turn left or right?
 - Should I go around the obstacle?
- This is often handled using:
 - Path planning algorithms
 - Artificial Intelligence
 - Reinforcement Learning

Motion control: Execute movement

- This converts decisions into motor commands.
- It ensures:
 - Smooth movement
 - Correct speed
 - Accurate turning
 - Trajectory tracking
- Even if planning is correct, bad motion control can make the robot crash.

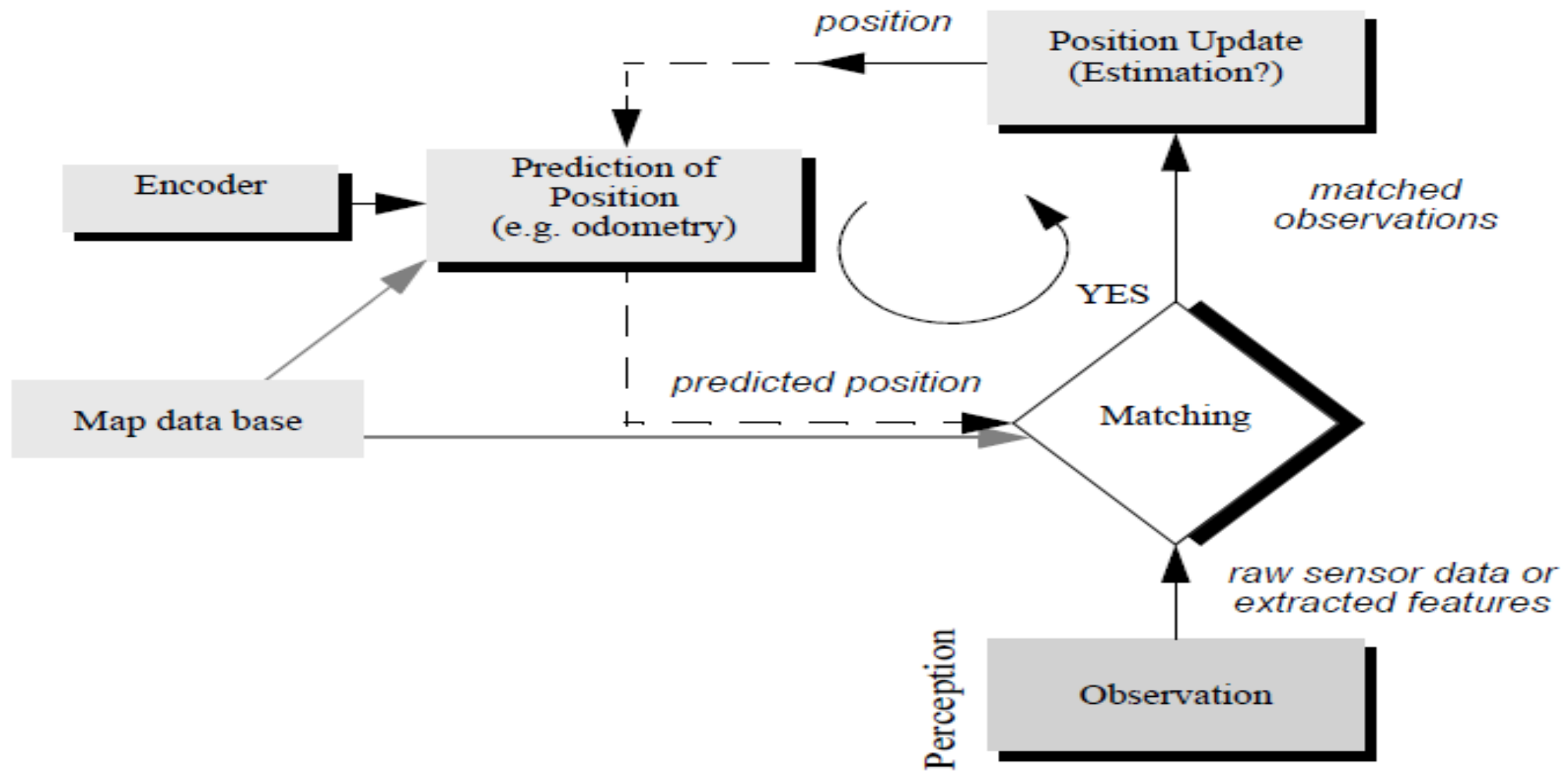


Figure 5.2
General schematic for mobile robot localization.

Explanation of above figure 5.2:

Step 1: Start with an initial position

- The robot begins with an initial guess of its position inside the map.
- (The map database stores a structured representation of the environment (walls, obstacles, landmarks, or occupancy probabilities) that the robot uses to compare and correct its estimated position.)

Step 2: Move the robot

- The robot moves using its wheels.

Step 3: Measure wheel rotation (Encoder step)

- The encoders are sensors that measure how much each wheel rotated.
- From this, the robot calculates:
 - How far it moved
 - How much it turned

Step 4: Predict the new position

- Using the wheel movement, the robot estimates its new position.
- This is only a **prediction**, not exact.

Step 5: Sense the environment (Observation step)

- The robot uses sensors (like LiDAR or camera) to look at:
- Walls
- Corners
- Obstacles
- Landmarks

Step 6: Compare with the map (Matching step)

- The robot checks: If I am really at this predicted position, should I see these walls and objects?
- It compares:
 - Predicted position
 - Sensor readings
 - Map database

Step 7: Check the error

- The robot checks if the sensor readings match the map.
- If they match well → good prediction
- If they don't match → position needs correction

Step 8: Correct the position (Update step)

- The robot adjusts its position estimate to reduce the error.
- Now it has a better estimate of (x, y, θ) .

Step 9: Repeat forever

- The robot keeps repeating:
- **Move → Predict → Sense → Compare → Correct → Repeat**

Reactive Navigation

- Reactive navigation is used in dynamic and unpredictable environments where pre-planned paths may not be effective.
- It allows an agent (e.g., a robot) to adapt and react quickly to environmental changes without relying on extensive prior knowledge or maps.
- **Example: Obstacle Avoidance in Mobile Robots**
- The robot **uses sensors** (such as ultrasonic, infrared, or LIDAR) to detect obstacles.
- It employs **reactive algorithms** to make **instantaneous decisions**, such as:
 - Adjusting speed
 - Changing direction
 - Performing evasive maneuvers

- Integration with Other Navigation Techniques: Reactive navigation can be combined with deliberative planning (which involves strategic decision-making).
- It can also work with learning-based approaches, where the system learns from experience to improve navigation.
- This combination provides a balance between immediate responses and long-term planning.
- Flexibility and adaptability: The agent interacts with a dynamic environment in real time.
- Common applications: Mobile robots, Autonomous vehicles, Other autonomous systems that require quick decision-making.

Localization and challenges of localization- Assignment 2 topic

- Please study from the text book I have uploaded in googleclassroom.
Module 3 &4 – same text book.

An error model for odometric position estimation

An error model for odometric position estimation

- Odometry means Estimate where the robot is by measuring how much the wheels have moved.

The robot's **pose** (position + orientation) is written as:

$$p = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

This simply means:

- x → robot's position in x-direction
- y → robot's position in y-direction
- θ → robot's orientation (heading angle)

So pose = (x, y, θ)

- For a **differential-drive robot**, If we know:
 - **Starting position**
 - **How much the robot moves in small time intervals**
- Then **we can find the new position by:**
 - **Adding up small movements step by step**
- This is called **integration of motion**
(or summing incremental distances).

At each small time interval Δt :

The robot moves:

$$\Delta x, \quad \Delta y, \quad \Delta \theta$$

Then we update:

$$x_{new} = x_{old} + \Delta x$$

$$y_{new} = y_{old} + \Delta y$$

$$\theta_{new} = \theta_{old} + \Delta \theta$$

Then repeat for next time step.

$$\Delta x = \Delta s \cos(\theta + \Delta\theta/2)$$

$$\Delta y = \Delta s \sin(\theta + \Delta\theta/2)$$

These equations calculate the **small change in position** of a differential-drive robot during one time step.

$$\Delta s$$

= small distance traveled by the robot during Δt

For differential drive:

$$\Delta s = \frac{\Delta s_r + \Delta s_l}{2}$$

$$\Delta\theta$$

= change in orientation during that time step

For differential drive:

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{L}$$

where:

- L = distance between wheels

- Δs_r : Distance traveled by the **right wheel** during the small time interval Δt
- Δs_l : Distance traveled by the **left wheel** during the same time interval

During the time step:

- Robot starts at angle θ
- Ends at angle $\theta + \Delta\theta$

Instead of using start or end angle,
we use the **middle angle**

$$\theta + \frac{\Delta\theta}{2}$$

This gives better accuracy.

If you move a distance Δs at angle θ :

- x movement = $\Delta s \cos\theta$
- y movement = $\Delta s \sin\theta$
- ..

After computing:

$$\Delta x, \Delta y, \Delta\theta$$

We update pose:

$$x_{new} = x_{old} + \Delta x$$

$$y_{new} = y_{old} + \Delta y$$

$$\theta_{new} = \theta_{old} + \Delta\theta$$

And repeat for next time step.

Thus we get the updated position p' :

$$p' = \begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = p + \begin{bmatrix} \Delta s \cos(\theta + \Delta\theta/2) \\ \Delta s \sin(\theta + \Delta\theta/2) \\ \Delta\theta \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \Delta s \cos(\theta + \Delta\theta/2) \\ \Delta s \sin(\theta + \Delta\theta/2) \\ \Delta\theta \end{bmatrix}. \quad (5.6)$$

By using the relation for $(\Delta s; \Delta \theta)$ of equations (5.4) and (5.5) we further obtain the basic equation for odometric position update (for differential drive robots):

$$p' = f(x, y, \theta, \Delta s_r, \Delta s_l) = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \frac{\Delta s_r + \Delta s_l}{2} \cos\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r + \Delta s_l}{2} \sin\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r - \Delta s_l}{b} \end{bmatrix}. \quad (5.7)$$

As we discussed earlier, odometric position updates can give only a very rough estimate of the actual position. Owing to integration errors of the uncertainties of p and the motion errors during the incremental motion $(\Delta s_r; \Delta s_l)$, the position error based on odometry integration grows with time.

- **The error propagation model for odometry.**

A robot estimates its position using **odometry** (wheel encoder measurements).

Robot pose:

$$p = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

where

- x = x position
- y = y position
- θ = orientation

But **odometry is not perfect.**

Reasons:

- wheel slip
- uneven floor
- encoder noise
- mechanical errors

So the robot position has **uncertainty**.

This uncertainty is represented using a **covariance matrix**.

The robot computes the next pose using wheel motions.

$$p' = f(p, \Delta s_r, \Delta s_l)$$

New pose = function of

- previous pose p
- right wheel motion Δs_r
- left wheel motion Δs_l

In simple words:

new position = old position + movement of wheels

Because:

- old pose has uncertainty
- wheel motion has uncertainty

So the new pose will also have uncertainty.

We must **propagate the uncertainty**.

For two random variables:

$$X_1, X_2$$

Their covariance matrix is:

$$\begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) \end{bmatrix}$$

So here:

$$X_1 = \Delta s_r$$

$$X_2 = \Delta s_l$$

The Equation (5.8)

$$\Sigma_{\Delta} = \text{covar}(\Delta s_r, \Delta s_l) = \begin{bmatrix} k_r |\Delta s_r| & 0 \\ 0 & k_l |\Delta s_l| \end{bmatrix}$$

Note:

Symbol	Description	Physical Meaning
Σ_{Δ}	Covariance matrix of wheel motion	Represents motion uncertainty
Δs_r	Distance traveled by right wheel	Measured from encoder
Δs_l	Distance traveled by left wheel	Measured from encoder
k_r	Right wheel error constant	Depends on motor noise, slip, surface
k_l	Left wheel error constant	Same as above

the covariance matrix Σ_p of the odometric position estimate. To do so, we assume that at the starting point the initial covariance matrix Σ_p is known. For the motion increment $(\Delta s_r; \Delta s_l)$ we assume the following covariance matrix Σ_Δ :

$$\Sigma_\Delta = \text{covar}(\Delta s_r, \Delta s_l) = \begin{bmatrix} k_r |\Delta s_r| & 0 \\ 0 & k_l |\Delta s_l| \end{bmatrix}, \quad (5.8)$$

where Δs_r and Δs_l are the distances traveled by each wheel, and k_r , k_l are error constants representing the nondeterministic parameters of the motor drive and the wheel-floor interaction. As you can see, in equation (5.8) we made the following assumptions:

- The two errors of the individually driven wheels are independent,²²
- The variance of the errors (left and right wheels) are proportional to the absolute value of the traveled distances $(\Delta s_r; \Delta s_l)$.

In matrix ,

$$\text{cov}(\Delta s_r, \Delta s_l) = 0$$

- Right and left wheel errors are assumed independent.
 - So no correlation between them.
 - If one slips, we assume it does not affect the other.
-
- Each wheel has independent noise
 - Noise increases with distance
 - Noise magnitude depends on constants k_r, k_l

Variance

$$\text{Var}(\Delta s_r) = k_r |\Delta s_r|$$

$$\text{Var}(\Delta s_l) = k_l |\Delta s_l|$$

- Each wheel has independent noise
- Noise increases with distance
- Noise magnitude depends on constants k_r, k_l

If wheel moves small distance → small chance of slip

If wheel moves large distance → more accumulated error

After finding uncertainty/covariance, then we propagate error to $p' = f(p, \Delta s_r, \Delta s_l)$

$$\Sigma_{p'} = \nabla_p f \Sigma_p \nabla_p f^T + \nabla_{\Delta r_l} f \Sigma_{\Delta} \nabla_{\Delta r_l} f^T$$

$\Sigma_{p'}$	Updated covariance matrix	Uncertainty of the new robot pose after movement
Σ_p	Previous covariance matrix	Uncertainty of the old robot pose before movement
Σ_{Δ}	Motion covariance matrix	Uncertainty in wheel movements (encoder errors, slip)
f	Motion model function	Equation that computes the new pose from the old pose and wheel movements
$\nabla_p f$	Jacobian of motion model w.r.t pose	Shows how errors in the previous pose affect the new pose
$\nabla_{\Delta r_l} f$	Jacobian of motion model w.r.t wheel motion	Shows how errors in wheel motion affect the new pose
$(\cdot)^T$	Transpose of a matrix	Used in covariance propagation to maintain correct matrix dimensions
$\nabla_p f \Sigma_p \nabla_p f^T$	First term of equation	Propagates previous pose uncertainty to the new pose
$\nabla_{\Delta r_l} f \Sigma_{\Delta} \nabla_{\Delta r_l} f^T$	Second term of equation	Adds new uncertainty caused by wheel motion

$\Sigma_{p'}$	Updated covariance matrix	Uncertainty of the new robot pose after movement
Σ_p	Previous covariance matrix	Uncertainty of the old robot pose before movement
Σ_{Δ}	Motion covariance matrix	Uncertainty in wheel movements (encoder errors, slip)
f	Motion model function	Equation that computes the new pose from the old pose and wheel movements
$\nabla_p f$	Jacobian of motion model w.r.t pose	Shows how errors in the previous pose affect the new pose
$\nabla_{\Delta r_i} f$	Jacobian of motion model w.r.t wheel motion	Shows how errors in wheel motion affect the new pose
$(\cdot)^T$	Transpose of a matrix	Used in covariance propagation to maintain correct matrix dimensions
$\nabla_p f \Sigma_p \nabla_p f^T$	First term of equation	Propagates previous pose uncertainty to the new pose
$\nabla_{\Delta r_i} f \Sigma_{\Delta} \nabla_{\Delta r_i} f^T$	Second term of equation	Adds new uncertainty caused by wheel motion

- summary

- 1 Robot computes new pose

$$p' = f(p, \Delta s_r, \Delta s_l)$$

- 2 But motion has uncertainty.

- 3 So we propagate uncertainty using

$$\Sigma_{p'} = F_p \Sigma_p F_p^T + F_\Delta \Sigma_\Delta F_\Delta^T$$

$$\Sigma_{p'} = \nabla_p f \Sigma_p \nabla_p f^T + \nabla_{\Delta r_l} f \Sigma_{\Delta} \nabla_{\Delta r_l} f^T$$

The robot motion equation is **non-linear**:

$$x' = x + \Delta s \cos(\theta + \Delta\theta/2)$$

$$y' = y + \Delta s \sin(\theta + \Delta\theta/2)$$

Because of **sin** and **cos**, we cannot propagate uncertainty directly.

So we linearize the function using derivatives → this gives **Jacobian matrices**.

Jacobian with Respect to Pose F_p ((Eq. 5.10))

$$F_p = \begin{bmatrix} 1 & 0 & -\Delta s \sin(\theta + \Delta\theta/2) \\ 0 & 1 & \Delta s \cos(\theta + \Delta\theta/2) \\ 0 & 0 & 1 \end{bmatrix}$$

Row 1	How pose errors affect x position
Row 2	How pose errors affect y position
Row 3	How pose errors affect orientation

How this matrix is obtained

It is computed from derivatives of the motion function:

$$F_p = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial \theta} \end{bmatrix}$$

JYO,AH,MEC

Jacobian with Respect to Wheel Motion

F_{Δ})(Eq. 5.11)

Column	Meaning
Column 1	effect of right wheel motion error
Column 2	effect of left wheel motion error

$$F_{\Delta} = \begin{bmatrix} \frac{1}{2} \cos(\theta + \Delta\theta/2) - \frac{\Delta s}{2b} \sin(\theta + \Delta\theta/2) & \frac{1}{2} \cos(\theta + \Delta\theta/2) + \frac{\Delta s}{2b} \sin(\theta + \Delta\theta/2) \\ \frac{1}{2} \sin(\theta + \Delta\theta/2) + \frac{\Delta s}{2b} \cos(\theta + \Delta\theta/2) & \frac{1}{2} \sin(\theta + \Delta\theta/2) - \frac{\Delta s}{2b} \cos(\theta + \Delta\theta/2) \\ \frac{1}{b} & -\frac{1}{b} \end{bmatrix}$$

Where

b = distance between wheels

$$\Sigma_{p'} = \nabla_p f \cdot \Sigma_p \cdot \nabla_p f^T + \nabla_{\Delta_{rl}} f \cdot \Sigma_{\Delta} \cdot \nabla_{\Delta_{rl}} f^T. \quad (5.9)$$

The covariance matrix Σ_p is, of course, always given by the $\Sigma_{p'}$ of the previous step, and can thus be calculated after specifying an initial value (e.g., 0).

Using equation (5.7) we can develop the two *Jacobians*, $F_p = \nabla_p f$ and $F_{\Delta_{rl}} = \nabla_{\Delta_{rl}} f$:

$$F_p = \nabla_p f = \nabla_p(f^T) = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial \theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\Delta s \sin(\theta + \Delta\theta/2) \\ 0 & 1 & \Delta s \cos(\theta + \Delta\theta/2) \\ 0 & 0 & 1 \end{bmatrix}, \quad (5.10)$$

$$F_{\Delta_{rl}} = \begin{bmatrix} \frac{1}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) - \frac{\Delta s}{2b} \sin\left(\theta + \frac{\Delta\theta}{2}\right) & , & \frac{1}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2b} \sin\left(\theta + \frac{\Delta\theta}{2}\right) \\ \frac{1}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2b} \cos\left(\theta + \frac{\Delta\theta}{2}\right) & , & \frac{1}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) - \frac{\Delta s}{2b} \cos\left(\theta + \frac{\Delta\theta}{2}\right) \\ \frac{1}{b} & & -\frac{1}{b} \end{bmatrix} \quad (5.11)$$

The details for arriving at equation (5.11) are

$$F_{\Delta_{rl}} = \nabla_{\Delta_{rl}} f = \left[\frac{\partial f}{\partial \Delta s_r} \frac{\partial f}{\partial \Delta s_l} \right] = \dots \quad (5.12)$$

$$\left[\begin{array}{cc} \frac{\partial \Delta s}{\partial \Delta s_r} \cos\left(\theta + \frac{\Delta \theta}{2}\right) + \frac{\Delta s}{2} \sin\left(\theta + \frac{\Delta \theta}{2}\right) \frac{\partial \Delta \theta}{\partial \Delta s_r} & , \frac{\partial \Delta s}{\partial \Delta s_l} \cos\left(\theta + \frac{\Delta \theta}{2}\right) + \frac{\Delta s}{2} \sin\left(\theta + \frac{\Delta \theta}{2}\right) \frac{\partial \Delta \theta}{\partial \Delta s_l} \\ \frac{\partial \Delta s}{\partial \Delta s_r} \sin\left(\theta + \frac{\Delta \theta}{2}\right) + \frac{\Delta s}{2} \cos\left(\theta + \frac{\Delta \theta}{2}\right) \frac{\partial \Delta \theta}{\partial \Delta s_r} & , \frac{\partial \Delta s}{\partial \Delta s_l} \sin\left(\theta + \frac{\Delta \theta}{2}\right) + \frac{\Delta s}{2} \cos\left(\theta + \frac{\Delta \theta}{2}\right) \frac{\partial \Delta \theta}{\partial \Delta s_l} \\ \frac{\partial \Delta \theta}{\partial \Delta s_r} & \frac{\partial \Delta \theta}{\partial \Delta s_l} \end{array} \right] \quad (5.13)$$

and with

$$\Delta s = \frac{\Delta s_r + \Delta s_l}{2} \quad ; \quad \Delta \theta = \frac{\Delta s_r - \Delta s_l}{b} \quad (5.14)$$

$$\frac{\partial \Delta s}{\partial \Delta s_r} = \frac{1}{2} \quad ; \quad \frac{\partial \Delta s}{\partial \Delta s_l} = \frac{1}{2} \quad ; \quad \frac{\partial \Delta \theta}{\partial \Delta s_r} = \frac{1}{b} \quad ; \quad \frac{\partial \Delta \theta}{\partial \Delta s_l} = -\frac{1}{b}, \quad (5.15)$$

we obtain equation (5.11).